SynthOmnicon: Typed Programming Language for Matter

March 20, 2026

Status: Phase 3a complete · Phase 3d complete (v0.4.0) · Phase 3b/3c pending · Phase 3e in progress (v0.4.4)

0.1 Vision

The Synthonicon is a **type system for matter** — not just chemistry. Its eleven primitives are type constructors spanning molecular, supramolecular, temporal, quantum, and topological domains. Its axioms are refinement constraints. HotSwap transitions are category morphisms. Retrodesign is type inference. The Varma probe embeds proof obligations. The phase diagram maps the metric geometry of the type space itself.

v0.4.0 revealed something deeper than expected: the primitive algebra is **domain-agnostic by construction**. When quantum particles and topological materials were encoded using the same eleven primitives, the algebra immediately produced physically correct results — non-Abelian anyons outranking -class invariants in tensor products, the K_trap→K_MBL universal cost (+2.303 nats) appearing identically across three different topological phases, the Ward dendrogram separating extended topological matter from point particles at d≈9.52 with no physics input. The framework did not simulate physics; it discovered it from the ordinal structure of the type system.

The goal of this track is to make that implicit structure **explicit and mechanically enforced** — moving from "axioms checked at runtime via string matching" toward "illegal states unrepresentable by construction," and from "phase boundaries detected empirically" toward "phase boundaries provable from the Kleisli metric."

The primitives are relational operators, not intrinsic attributes. Every element of the tuple describes a constraint between entities or a capacity for interaction. F (fidelity) is reliability of constraint satisfaction relative to a competitor — there is no intrinsic F, only F relative to a context. K (kinetic) is a barrier to rearrangement, implying at least two states. Ω (topological protection) is protection against perturbations, meaningless without an environment. Γ (grammar) is partner selection logic by definition. This is not a philosophical gloss; it is a **type-system requirement**: you cannot assign F, K, Γ, or Ω without specifying an interaction context. A synthon tuple without a context is a description of interaction potential, not isolated being.

The algebra enforces this. Every operation in `meet / join / tensor / path / lift`

`/ pipeline` requires at least one additional operand. `tensor(s, s)` computes mutual information between two systems. `path(src, dst)` requires two endpoints. `lift(s, critical)` requires a named target context and is blocked by a relational gate (`F >=` F_h). There are no unary information generators. The algebra cannot process "nothing but the object." This is the deeper reason the type system works: **it is a calculus of relations, and relations require at least two terms**.

Furthermore, the algebra is systematically asymmetric — `path(A->B) != path(B->A)`, the F-floor ratchet is directed, `lift` has no inverse. This places the framework in the tradition of structural realism rather than symmetric relational theories: the world's causal structure is relational but ordered, and the ordering is the load-bearing part.

A design program in the finished language looks like:

```
# design.syn
version: "1.0"
start: soai_pyrimidyl_autocatalytic_cycle
do:
  - join: proline_aldol_cycle
  - lift: critical
  - path: varma_qxy_reference
    xi_tolerance: 1.5
  - assert:
      expr: phi_c_score > 0.70
      message: "Must approach Phi_c before ensemble step"
  - tensor: db24c8_dialkylammonium_pseudorotaxane
output:
  format: text
  save: result.json
```

`syncon run design.syn` evaluates this as a proper monadic do-block with typed effects, inline proof obligations, and full step tracing.

0.2 Current State (v0.4.0)

0.3 Phase 3a — Monadic Foundation

Target: `syncon run design.syn` works end-to-end.

0.3.1 3a.1 `synthomnicon/monad.py` — SynthonM

`SynthonM[A]` is the monad transformer stack:

```
SynthonM[A] \cong WriterT[float] (StateT[Context] (MaybeT
   Identity)) A
```

Concretely, a `SynthonM[Synthon]` carries:

Monad primitives:

Component	Status	Notes
Eleven-primitive tuple	complete	;T;R;P;F;K;G;Γ;Φ;S;Ω⟩ − −−Ω*optional*, *defaultsNoneforclassical*
Axioms 1–7	complete	Runtime; Axiom 6 discrete/continuous; Axiom 1 is quantum boundary detector
Algebra (meet/join/tensor/lift/path/distance)	complete	`algebra.py`; all commands wired to `syncon`
DesignPipeline (Writer+Maybe)	complete	Fluent interface + `bind`/`mzero`/`mplus` via SynthonM
F-floor HotSwap ratchet	validated	6/6 CB[7] experimental match
Varma probe (8 mechanisms)	validated	Factors 1–5 QXY · Factor 6 steric-cliff · Factor 7 Frank · Factor 8 quantum criticality
`.syn` DSL runner	complete	`syncon run design.syn` — `syn_runner.py` ~380 lines
`SynthonM` monad type	complete	`monad.py` — WriterT+StateT+MaybeT stack
`DesignStrategy` type + combinators	complete	`strategy_then`, `strategy_or`, `optimize`
Quantum primitive extensions	complete	T_braid · K_MBL · Γ_/QUANTUM · Ω (TopoIndex) — Phase 3d
Quantum catalog (8 synthons)	complete	`domains/quantum/` — particles + topological matter, all Ω assigned
Tuple-space phase diagram	complete	`phase_diagram.py` — Ward clustering + MDS + `syncon phase-diagram`
Refinement types (Pydantic v2)	pending	Phase 3b target
`from_disorder` KineticCharacter constructor	pending	Phase 3b — K_MBL cannot use `from_barrier`
`context.irreversible` in SynthonM	pending	Phase 3b — Γ_ dissipative flag propagation
Kleisli category (formal)	pending	Phase 3c target
Z3/SMT retrodesign	pending	Phase 3c target
Lean 4 axiom formalization	pending	Phase 3c long-term
Holographic dimensionality D_holo	complete	Phase 3e — `Dimensionality.HOLOGRAPHIC`; `ads_cft_boundary` synthon registered
Phase transitions as morphisms	complete	Phase 3e — `synthomnicon/morphism.py`; `syncon transition` CLI; Kleisli arrow classification

Effect	Carrier	Meaning
MaybeT	`value: Optional[A]`	Computation may fail (BLOCKED/ERROR)
WriterT	`cost: float`	Accumulated $\Delta\xi$_CP across all steps
StateT	`context: Context`	F-floor, criticality gate, step count
Log	`log: List[StepRecord]`	Full step trace for human inspection

```
return_(a)          # Pure: lift value, zero cost
m.bind(f)           # Sequence: run m, feed into f, merge
   effects
m >> f              # Operator alias for bind
mzero()             # Failed computation
m1.mplus(m2)        # Try m1; if None, use m2
m1 | m2             # Operator alias for mplus
```

Monadic lifts wrap each algebra operation:

```
join_m(name)              # algebra.join -> SynthonM
meet_m(name)              # algebra.meet -> SynthonM
tensor_m(name, lambda)        # algebra.tensor -> SynthonM
lift_m(target)            # _LIFT_MAP[target] -> SynthonM
path_m(name, \xi_tol)      # algebra.find_path -> SynthonM
assert_m(pred, msg)       # inline proof obligation -> SynthonM
```

Strategy type:

```
DesignStrategy = Callable[[Synthon], SynthonM[Synthon]]

# Sequential composition
strategy_then(s1, s2)  # s1 >> s2

# Alternative / fallback
strategy_or(s1, s2)    # s1 <|> s2

# First-success search
optimize(synthon, [s1, s2, s3])  # asum over strategy list
```

0.3.2 3a.2 `synthomnicon/syn_runner.py` — .syn DSL Evaluator

Parses `.syn` YAML files and evaluates them as `SynthonM` pipelines.

Step dispatch table:

`assert` expression grammar (safe dispatch, no `eval`):

Step key	Monadic lift	Example
join	join_m(name)	- join: proline_aldol_cycle
meet	meet_m(name)	- meet: nitroso_radical
tensor	tensor_m(name, lambda)	- tensor: db24c8\n lambda: 0.4
lift	lift_m(target)	- lift: critical
path	path_m(name, ξ)	- path: varma_qxy\n xi_tolerance: 1.5
assert	assert_m(pred)	- assert:\n expr: phi_c_score > 0.70
bind	named strategy	- bind: my_strategy

```
phi_c_score > N        -> run varma probe, check score
phi_c_score >= N       -> same
fidelity == F_hbar     -> check synthon.fidelity
topology == T_bowtie   -> check synthon.topology
criticality_phase == Phi_c  -> check enum value
axiom6_satisfied       -> run AxiomValidator.
   validate_axiom6_temporal_grounding
gd_degeneracy == X     -> run probe, check gd_degeneracy_type
reset_type == discrete -> check grounding["reset"]["type"]
reset_type == continuous  -> same
```

0.3.3 3a.3 CLI: syncon run

```
syncon run design.syn
syncon run design.syn --format json
syncon run design.syn --dry-run    # parse + validate without
   executing
syncon run design.syn --save result.json
```

Output (text mode):

```
SynthOmnicon Run: design.syn
  Start: soai_pyrimidyl_autocatalytic_cycle

  1. [PASS]  join(proline_aldol_cycle)  Delta\xi=+0.000 nat
  2. [BLOCKED]  lift(critical)  --- F floor not met (F_eth <
   F_hbar required)
  3. [PASS]  assert(phi_c_score > 0.70)  --- ASSERT_FAIL:
   score=0.380

  Total Delta\xi_CP: +0.000 nat  |  Steps: 3  |  FAILED at
   step 2
```

0.4 Phase 3b — Refinement Types

Target: illegal states are unrepresentable, not just runtime-rejected.

0.4.1 3b.1 Pydantic v2 validators on Synthon

Move all 7 axioms into `@model_validator(mode='after')` decorators. Construction of an invalid Synthon raises `ValidationError`, not a `grounding_failure` flag.

```
class Synthon(BaseModel):
    fidelity: Fidelity
    topology: Topology
    # ...

    @model_validator(mode='after')
    def axiom1_fidelity_floor(self):
        if self.topology == Topology.CYCLIC_BOWTIE:
            if self.fidelity == Fidelity.LOW:
                raise ValueError("Axiom 1: T_ requires F >=
  F_eth")
        return self
```

0.4.2 3b.2 Ordered primitive types

```
class FidelityGe(Annotated[Fidelity, Ge(Fidelity.MEDIUM)]):
    """F >= F_eth --- satisfied by F_eth or F_hbar"""

CriticalityCandidateSynthon = Synthon[fidelity=FidelityGe]
```

Retrodesign returns `list[CriticalityCandidateSynthon]` for Φ_c targets — caller knows statically that every leaf has F ≥ F_eth.

0.4.3 3b.3 Protocol classes for domains

```
class TemporalSynthon(Protocol):
    """Structural subtype requiring D_inf and grounding["reset
  "] block."""
    @property
    def dimensionality(self) -> Literal[Dimensionality.
  TEMPORAL]: ...
    @property
    def grounding(self) -> dict:
        assert "reset" in self.grounding
```

`trajectory validate` and `retrodesign` for D_∞ targets return `TemporalSynthon` so the type checker catches missing reset blocks before execution.

0.5 Phase 3c — Category Theory Embedding

Target: the algebra is provably correct and retrodesign is constraint-satisfaction.

0.5.1 3c.1 Explicit Kleisli category

Document the category formally:

- **Objects**: Synthons (equivalence classes of valid ten-tuples)
- **Morphisms**: HotSwap transitions `f: A -> B` with cost `Delta$\xi$_CP(f) in >=0`
- **Identity**: trivial self-swap, cost 0
- **Composition**: `find_path` (BFS composes morphisms; cost = sum of $\Delta\xi$_CP per hop)
- **Enrichment**: over `(>=0, +, 0)` — a Lawvere metric space
- **Monoidal product**: `tensor` (bifunctor; F-bottleneck as monoidal unit constraint)
- **Natural transformations**: `lift_*` functions (lift_to_temporal, etc.)

Add `synthomnicon/category.py` with formal composition and identity checks.

0.5.2 3c.2 Z3/SMT for retrodesign

Replace BFS in `retrodesign.py` with Z3 constraint satisfaction:

```
# target: \langleD_inf; T_; R_; P_DA; $F_{\hbar}$; ...;
   Phi_sub; 1:1\rangle
# find all catalog entries satisfying axioms 1,2,4,6 and
# compositionally derivable from the target tuple
solver = z3.Solver()
# encode the ten-tuple as Z3 bitvectors
# encode axioms as Z3 constraints
# enumerate all satisfying assignments
```

This gives **complete** retrodesign (no false negatives from BFS depth limit) and opens constraint-solving for multi-target design.

0.5.3 3c.3 Lean 4 axiom formalization (long-term)

Formalize the seven axioms as Lean 4 theorems. The Python implementation becomes a "trusted kernel" whose correctness is guaranteed by the proof. Key lemmas:

- `axiom1_preserved_by_hotswap`: HotSwap preserves F $\geq$ F_eth for T_ entries
- `join_idempotent`: `join(s, s) = s`
- `meet_join_absorption`: `meet(s, join(s, t)) = s`
- `tensor_fidelity_bottleneck`: `tensor(s1, s2).F = min(s1.F, s2.F)`

0.6 File Index

0.7 Design Decisions

Why WriterT + StateT + MaybeT (not ExceptT)? ExceptT short-circuits on the first failure and discards subsequent state. MaybeT propagates the accumulated cost and

File	Phase	Description
`synthomnicon/monad.py`	3a	SynthonM stack, monadic lifts, DesignStrategy
`synthomnicon/syn_runner.p`	3a	.syn YAML DSL evaluator
`synthomnicon/cli.py`	3a/3d/3e	`syncon run` · algebra commands · `syncon phase-diagram`
`synthomnicon/models.py`	3d	T_braid · K_MBL · QUANTUM/DISSIPATIVE grammar · TopoIndex
`synthomnicon/algebra.py`	3d	Ω lattice ops · eleven-dimensional `tuple_distance`
`synthomnicon/varma_probe.`	3d	Factor 8 — quantum criticality block
`synthomnicon/domains/quan`	3d	8 canonical quantum/topological synthons
`synthomnicon/phase_diagra`	3e	Ward clustering · MDS · `PhaseDiagram` dataclass
`synthomnicon/morphism.py`	3e	`TransitionMorphism` · `find_transition()` · Kleisli arrow classification · `syncon transition`
`synthomnicon/category.py`	3c	Kleisli category formal definition (pending)
`synthomnicon/typed.py`	3b	Pydantic v2 Synthon, refinement types (pending)

log even on failure, preserving the full trace. This matches the existing `DesignPipeline` semantics where BLOCKED steps are logged but execution records continue.

Why YAML for `.syn` (not a custom syntax)? YAML is already parsed everywhere in the project; it's readable and diffable; LLMs generate it reliably. A custom syntax would require a parser that buys nothing concrete at this stage. The DSL can migrate to a typed syntax (like Dhall or a subset of Haskell do-notation) in Phase 3b once the semantics are stable.

Why safe dispatch for `assert` expressions (not `eval`)? `eval()` with arbitrary Python is a security surface and makes the language unpredictable. The predefined assertion grammar is a subset of useful predicates; any pattern not in the dispatch table raises a clear `UnknownAssertion` error pointing to the docs.

Why not rewrite in Haskell? The Python catalog, CLI, and grounding layer are the validated, empirically-anchored core. Rewriting loses that incrementally. The right path is to formalize the semantics in Python first (Phase 3a/3b), then extract the formal core to Lean 4 (Phase 3c) as a proof of correctness — not a replacement.

0.8 Phase 3d — Quantum Primitive Extensions (v0.4.0)

Motivation. Encoding five quantum particles (photon, proton, electron, spin, qubit) through the LLM agent using only the classical primitive set revealed seven structural gaps — places where the framework failed to encode known physics, and where each failure pointed at a real but undescribed class of physical system. The gaps were filled by adding four new primitive values and one new primitive.

0.8.1 New primitive values

Primitive	New value	Encodes	Physical systems
T	`T_braid`	Anyonic/braided exchange statistics	Fractional QHE (ν=1/3, ν=5/2), Kitaev honeycomb B-phase
K	`K_MBL`	Many-body localization — disorder-frozen	Disordered quantum magnets, Aubry-André cold atoms
Γ operator	`DISSIPATIVE` (Γ_)	Irreversible information loss	Lindblad open systems, quantum Zeno effect
Γ tier	`QUANTUM`	Superposition-preserving (Toffoli semantics)	Quantum AND-gates, fault-tolerant circuits

K_MBL lattice position: ordinal 0, below K_trap (ordinal 1). MBL is more kinetically arrested than any energy-barrier trap — the system cannot relax even with unlimited

energy input (the many-body eigenbasis structure prevents thermalization). `meet(K_MBL, K_fast) -> K_MBL`; `tensor(K_MBL, anything) -> K_MBL`.

T_braid tensor rule: `tensor(T_braid, T_braid) -> T_braid`. Two braided systems form a larger braided system; anyonic statistics do not network-promote like spatial structures. `T_braid  T_linear = ⊥` (the unresolvable conflict that motivated the primitive).

0.8.2 New primitive Ω (11th)

`TopoIndex` encodes the symmetry class of topological protection, based on the Altland–Zirnbauer / K-theory (10-fold way) classification:

```
Omega_0  (TRIVIAL)     --- No topological protection;
   classical systems
Omega_Z  (Z_CLASS)     ---  winding number: Kitaev chain, SSH
   model, 1D p-wave
Omega_Z2 (Z2_CLASS)    ---  invariant: HgTe/CdTe QWs, BiSe (
   class AII/DIII)
Omega_C  (CHERN)       --- Integer Chern number: IQH, Chern
   insulators (class A)
Omega_NA (NON_ABELIAN) --- Non-abelian anyons: nu=5/2 FQH,
   Kitaev honeycomb
```

Protection-strength ordinal: TRIVIAL(0) < Z2(1) < Z(2) < CHERN(3) < NON_ABELIAN(4).

In `meet`: lower protection propagates (conservative guarantee). In `join` and `tensor`: higher protection propagates (strongest class dominates). In `tuple_distance`: `Omega` weight = 0.7 (highest categorical penalty — a Chern insulator and a trivial metal are categorically different systems).

0.8.3 Factor 8 — Quantum criticality in the Varma probe

The spin singlet analysis showed that the Varma probe scored 0.0 for spin despite `G_aleph`, because all five classical factors require `D_inf`. Factor 8 fires on the orthogonal pattern:

```
G_aleph + F_hbar + K_trap + notD_inf  ->  quantum criticality
   (TFI/heavy_fermion class)
```

Weight 0.20. Falsifiable prediction: $\chi(T \to 0) \sim T^{-\gamma}$. The spin singlet scores 0.20 with universality class "quantum_criticality (TFI/heavy_fermion)", triggering `gd_degeneracy_type = "quantum_ground_state"` and Axiom 5 satisfaction.

0.8.4 Type-theoretic implications

The new primitives expose three open questions for Phase 3b:

1. **Ω is the first non-enum primitive candidate**: it has a natural integer (protection strength) representation in addition to its categorical (symmetry class) representation. Phase 3b refinement types could encode `Omega_NA >= Omega_Z` as a Pydantic `Ge` constraint.
2. **K_MBL violates `from_barrier` semantics**: disorder-induced localization cannot be assigned from a single $\Delta G^{\ddagger}$ value. The `from_barrier` classmethod correctly omits K_MBL, but a new `from_disorder` classmethod is implied.

3. Γ__**(DISSIPATIVE) introduces irreversibility into the grammar**: the existing `SynthonM` monad tracks cost accumulation (WriterT) but has no irreversibility primitive. A dissipative synthon flowing through a `SynthonM` pipeline should perhaps trigger a `StateT` flag: `context.irreversible = True`.

0.9 Phase 3e — Tuple-Space Phase Detection (v0.4.0)

Target: the framework discovers its own phase boundaries from the metric geometry of the type space.

0.9.1 3e.1 `synthomnicon/phase_diagram.py` — PhaseDiagram module

`PhaseDiagram` is a dataclass computed from the full N×N pairwise `tuple_distance` matrix over a named set of synthons:

```
@dataclass
class PhaseDiagram:
    synthon_names: List[str]
    distance_matrix: np.ndarray        # NxN symmetric
    candidates: List[PhaseBoundary]    # ranked boundary list
    linkage_matrix: np.ndarray         # Ward linkage (scipy
   format)
    mds_coords: np.ndarray             # Nx2 MDS embedding
    factor8_flags: Dict[str, bool]     # Factor-8 trigger per
   synthon
    omega_values: Dict[str, str]       # \Omega class per
   synthon
    k_trap_flags: Dict[str, bool]      # K_trap flag per
   synthon
```

`build_phase_map(synthon_names=None, catalog=None) -> PhaseDiagram` constructs the full object from the catalog in one call.

Phase boundary detection algorithm:

1. Compute N×N distance matrix via `tuple_distance`.
2. Run Ward hierarchical clustering on the upper triangle.
3. Extract consecutive height differences in the merge sequence.
4. Rank gaps as Major (top 33%), Intermediate (middle), Minor (bottom 33%).
5. The primary boundary (largest gap, d ≈ 9.52 for the 8-synthon quantum catalog) is the most semantically significant cluster split.

```
from synthomnicon.phase_diagram import build_phase_map
pd = build_phase_map()
pd.print_report()
pd.plot(save_path="diagram.png")
```

0.9.2 3e.2 `syncon phase-diagram` CLI command

```
# All synthons in catalog:
syncon phase-diagram

# Subset by name:
syncon phase-diagram photon proton electron
   kitaev_chain_majorana

# Save rendered figure:
syncon phase-diagram --save diagram.png

# Skip matplotlib (text + JSON only):
syncon phase-diagram --text-only

# JSON output for scripting:
syncon phase-diagram --format json
```

Output (text mode): distance matrix · phase boundary table (gap, label, threshold d) · Factor-8 cluster summary · K_trap/Ω membership list.

0.9.3 3e.3 MDS embedding — metric geometry of the type space

The 2-D MDS projection uses classical (metric) MDS via eigendecomposition of the double-centred Gram matrix:

```
B = -¡ H D^2 H          where H = I - (1/N)
B ~= V \Lambda V         (top-2 eigenpairs)
X = V \Lambda^{1/2}      (Nx2 coordinates)
```

No sklearn dependency — pure `numpy.linalg.eigh`. Stress is not reported; the Ward dendrogram is the primary clustering artifact.

Key observation from the 8-synthon quantum map:

- – The primary boundary at d ≈ 9.52 separates two branches with no physics input:**Branch A (extended topological):** kitaev_chain_majorana · fqh_moore_read · topological_insulator_bi2se3
  - – **Branch B (point particles):** photon · proton · electron · spin_singlet · qubit_logical
- Within Branch A, fqh_moore_read (T_braid + Ω_NA) separates from the -class pair at d ≈ 4.3.
- Within Branch B, proton and electron cluster at d ≈ 1.8 (charge sign only); qubit separates last at d ≈ 3.2.

The framework isolated the topological phase boundary by ordinal arithmetic alone — no Hamiltonian, no symmetry group, no physical input beyond primitive assignments.

0.9.4 3e.4 Universality track prediction

The perturbation sweep `K_trap -> K_MBL` produces $\Delta\xi = +2.303$ nats [HIGH] for spin_singlet, kitaev_chain_majorana, and fqh_moore_read — three synthons with different T

and Ω values, all producing the same thermodynamic cost. This is a **framework prediction**: the cost of entering MBL from a coherent gap-protected state is independent of the specific topological invariant.

The MDS map should display this as three parallel trajectories (same displacement vector, different starting positions) — this is the "universality track" that Phase 3e will make visually and algebraically explicit.

Falsification path: thermodynamic integration or quench spectroscopy on Kitaev-chain vs FQH vs spin-singlet systems should show the same entropy cost to disorder-freeze the state.

Contrasting result: topological_insulator_bi2se3, `K_slow -> K_moderate` $= -0.847$ nats [MEDIUM]. Improving surface mobility *reduces* thermodynamic cost — gapless surface states, not a gapped condensate. The sign flip is structural (K ordinal direction reverses semantics for metals vs insulators).

0.9.5 3e.5 Phase 3e stress test agenda

Three classes of objects identified by GPT-4 review as stress tests for the tuple-space phase detection:

Phase transitions as morphisms complete (v0.4.4) `synthomnicon/morphism.py` implements `TransitionMorphism` and `find_transition(src, dst, catalog)`. A transition A→B is classified as:

- **2nd order** — HotSwap path exists through Φ_c intermediates (continuous; `syncon transition` finds the path and reports forward/reverse costs and Φ_c intermediates)
- **1st order** — No path (D/T structural conflict); virtual Kleisli arrow with infinite primitive cost; latent heat ≈ barrier height between D/T classes

Key asymmetry result: `topological_insulator_bi2se3 -> synthon_Fermi_liquid` is an asymmetric 1st-order morphism — the reverse path (Fermi liquid → TI, fidelity *increases*) is permitted; the forward path (TI → Fermi liquid, fidelity *decreases*) is blocked by the F-floor. Topological protection encodes as morphism irreversibility. `syncon transition SRC DST` CLI registered.

Floquet synthons (periodic drive) complete (v0.4.4) `floquet_chern_insulator` ($D_\triangle$, $T_{\uparrow\downarrow}$, K_{trap}, Ω_C, $\Gamma_\rightarrow$(SEQUENTIAL)) and `time_crystal_dtc` (D_∞, $T_{\bowtie}$, K_{MBL}, Ω_Z) registered. $d(\text{floquet\_chern}, \text{time\_crystal}) = 4.2$ — same $G_{\rightthreetimes}/F_{\eth}/\Phi_{\text{sub}}$ floor, differing in $D/T/K$ and topological class (Ω_C vs. Ω_Z). $\Gamma_\rightarrow$(SEQUENTIAL) encodes the stroboscopic Floquet operator; K_{MBL} encodes the disorder-localization required for DTC stability against Floquet heating.

Gauge theories (loop variables) complete (v0.4.4) `z2_lattice_gauge_toric_code` registered: $\langle D_\triangle; T_{\uparrow\downarrow}; R_\Leftrightarrow; P^{\text{sym}}_{\pm}; F_\hbar; K_{\text{trap}}; G_\aleph; \Gamma_\wedge(\text{QUANTUM}); \Phi_{\text{sub}}; \Omega_{Z_2}\rangle$. Key result: $d(\text{z2\_toric}, \text{fqh\_moore\_read}) = 3.90$ at path cost = **0.000 nat**. Zero-cost path at non-zero tuple distance = same thermodynamic universality class, different topological invariant. $T_{\uparrow\downarrow}$ (Wilson loop holonomy), $R_\Leftrightarrow$ (loop variable recognition), $G_\aleph$ (global stabilisers) encode the gauge-theory structure without a new primitive.

0.10 Algebraic Operations Reference

Runnable companion: ***TENSOR_OPS_DEMO.py*** *— 18 worked examples,* ***python TENSOR_OPS_DEMO.py -section <op>****.*

The synthon tuple space is a product of bounded lattices. Seven operations act on it. Each maps cleanly to a concept from tensor mathematics, category theory, or constructive logic — not as a metaphor but as a precise structural translation.

0.10.1 Operation Table

Operation	Signature	Category-theoretic reading	SynthonM lift
`meet(s, s)`	`Synthon^2 -> Synthon {⊥}`	Categorical product; infimum in product lattice	`meet_m(name)`
`join(s, s)`	`Synthon^2 -> Synthon {⊥}`	Coproduct; supremum; F-floor raised in WriterT	`join_m(name)`
`tensor(s, s, lambda)`	`Synthon^2 x [0,1] -> Synthon`	Bifunctor; ξ_tensor $= \xi + \xi - \lambda \cdot$I(s,s)	`tensor_m(name, lambda)`
`lift(s, target)`	`Synthon -> Synthon`	Functor between domain categories	`lift_m(target)`
`path(src, dst)`	`Synthon^2 -> PathResult`	Geodesic in the Kleisli enriched category	`path_m(name, ξ_tol)`
`assert(pred)`	`(Synthon -> bool) -> SynthonM`	Inline proof obligation; does not alter value	`assert_m(pred, msg)`
decomp	see below	Inversion, projection, factorisation	`cofactor_m`, `factor_m`, `project_m`

0.10.2 ğ1 · meet () — Greatest Lower Bound

Primitive rules:

- *Categorical* (D, T, R, P, Γ): identity required; mismatch → ⊥
- *Ordered* (F, K, G, Ω): min — most conservative
- Φ: **Φ_c is an absorbing element** — Φ_c Φ_sub = Φ_c

The absorbing Φ_c is a co-Heyting top: not the usual infimum but a constraint forcing the result into the critical sub-lattice. The algebra cannot ”average away” a phase transition.

Key examples (ğ1 in demo):

Input	Output	Insight
`meet(Hv1_human_open, AtHv1_primed)`	PASS · Φ_c propagates	Cross-species water-chain conservation proved algebraically
`meet(Hv1_human_open, 2GBI_inhibitor)`	⊥· T-conflict + P-conflict	Conflict *is* the answer: inhibitor occludes, does not merge
`meet(cooper_pair, topological_insulator)`	PASS on Ω, ⊥on D/P/Γ	Ω_Z Ω_Z = Ω_Z (AZ classification ordinal)

0.10.3 ğ2 · join () — Least Upper Bound / Design Target

Primitive rules:

- *Categorical*: identity required; mismatch → ⊥
- *Ordered* (F, K, G, Ω): max — most demanding; F-floor ratchet raised
- Φ: **Φ_c is join-dominant** — Φ_c Φ_sub = Φ_c

join is the **design target**: the minimal synthon that both inputs can inject into. In module theory: the pushout. The F-floor ratchet implements the *directed* nature of commitment — once a fidelity obligation is raised, it cannot be lowered by any downstream operation.

Key examples (ğ2 in demo):

Input	Output	Insight
`join(Hv1_human_open, PsHv1_constitutive)`	PASS · Φ_c propagates, F→$F_\hbar$	Gymnosperm channel is algebraically subsumed
`join(2GBI_inhibitor, HIF_inhibitor)`	⊥· T-conflict	No common scaffold — the incompatibility is categorical, not quantitative
`join(imatinib, GNF-2)`	⊥· T-conflict reveals bottleneck	Dual-mechanism ABL design: T_ ∩T_branched = ∅

0.10.4 ğ3 · tensor (⊗) — Bifunctor / Co-Assembly Prediction

ξcost formula:

$$\xi_{\text{tensor}} = \xi_1 + \xi_2 - \lambda \cdot I(s_1, s_2)$$

where $I(s_1, s_2) \approx \frac{\text{primitive matches}}{7} \times \min(\xi_1, \xi_2)$ and $\lambda \in [0, 1]$ is the mutual-information discount (shared structure reduces assembly cost).

Primitive rules:

- D: union of domain sets

- T: **topology promotion** (cage > network > hub > bowtie > linear); same → same; `T_braid ⊗T_braid -> T_braid`
- F, K: min (bottleneck propagates)
- G: max (coarsest scale controls)
- Φ, Ω: join-dominant (criticality and topological protection propagate)

Critical semantic boundary:

> tensor predicts **co-occupancy** (the two-particle Hilbert space). A **bound state** requires `tensor` *then* `meet(binding_potential_synthon)` to acquire T_bowtie. This is the exciton theorem: `electron ⊗hole -> T_| ⊗T_| = T_|`; the Coulomb interaction that produces T_ is a *separate* meet step.

Key examples (ğ3 in demo):

Input	Output	Insight
`tensor(electron, hole, lambda=0.5)`	PASS · T stays LINEAR · P→DONOR_ACCEPTOR	bound state ≠ tensor product; Coulomb binding is a meet
`tensor(phonon_acoustic, magnon, lambda=0.4)`	PASS · F→F_ℓ · G→G_	magnetoelastic polaron; λ encodes spin-phonon coupling g_kq
`tensor(majorana, majorana, lambda=0.3)`	PASS · T_braid preserved · Ω_NA preserved	topological qubit: braid statistics do not network-promote

Special T_braid rule: `T_braid ⊗T_braid -> T_braid`. The braid group $B_n \otimes B_m \subseteq B_{n+m}$ — anyonic statistics compose into a larger braid system, never into a network.

0.10.5 ğ4 · lift — Natural Transformations Between Domain Categories

Three lifts are implemented as functors between domain categories:

Lift	Source → Target	Key primitive changes	Cost
`lift_to_temporal`	C_mol → C_temporal	D→D_∞, T→T_, R→R_, Γ→Γ_→(SEL)	0.0 nats
`lift_to_spatial`	C_mol → C_crystal	D→D_, T→T_, G escalates to MESOSCALE	0.0 nats
`criticality_lift`	C_sub → C_critical	Φ_sub→Φ_c	**+2.303 nats**

Criticality lift cost = 2.303 nats is the Shannon entropy of a binary phase transition: $\log_e(10)$, one decade of probability mass transfer. It is the primitive-space Landauer bound: the minimum cost of acquiring an order parameter.

Gate: `criticality_lift` requires F $\geq F_\hbar$. Blocked if F $< F_\hbar$. This is the relational gate that prevents phonons (F_ℓ) from being criticality-lifted — they are not the order parameter.

Asymmetry: lift has no inverse. The F-floor ratchet in SynthonM is monotone: `join` and `lift` raise the floor; no operation lowers it. This encodes thermodynamic irreversibility as a monad law.

Category-theoretic translations:

- `lift_to_temporal` $\cong loop{-}spacefunctor\Omega$: $sendsapointedspace(bindingevent)toitsloopspace(ca$ $classifyingspacefunctorB(G) : theSBUistheclassifyingobjectforthecrystal'spackingsymmetr$
- `criticality_lift` $\cong the''phaseenrichment''functorfromagenericcategorytoa\Phi\_c{-}structuredcatego$

0.10.6 ğ5 · path — Geodesic in the HotSwap Kleisli Category

Category:

- Objects: synthons (equivalence classes of 10-tuples)
- Morphisms: HotSwap transitions $f : A \to B$ with cost $\Delta\xi_{\text{CP}}(f) \geq 0$
- Enrichment: over $(\mathbb{R}_{\geq 0}, +, 0)$ — a **Lawvere metric space**
- Composition: BFS path; total cost additive

Hard topological gates (path blockers):

- D-class must be invariant along the path
- T-class must be invariant along the path

A **blocked path** = a T- or D-class boundary = **1st-order-like transition** (discrete jump, latent cost > 0). A **found path** = 2nd-order-like transition (continuous deformation within the same homotopy class).

Key examples (ğ5 in demo):

Source → Target	Status	Reason	Physical reading
`AtHv1_silent -> AtHv1_primed`	BLOCKED	T_∈ ≠ T_	Mechanical priming is a discrete topology jump, not continuous deformation
`2GBI_inhibitor -> HIF_inhibitor`	BLOCKED	T_∈ ≠ T_|	Scaffold evolution d=1.5 nats; SAR cannot bridge this
`topological_insulat -> electron`	BLOCKED	D + T mismatch	Topological gap = blocked path; bulk-boundary correspondence as monad law

Asymmetric morphisms (from Phase 3e): `path(TI -> Fermi liquid)` and `path(Fermi`

`liquid -> TI`) are both blocked, but the F-floor ratchet means the reverse costs more. Topological protection encodes as **morphism irreversibility**.

0.10.7 ğ6 · SynthonM — Monad Transformer Stack

```
SynthonM[A] \cong  WriterT[>=0] (StateT[Context] (MaybeT
   Identity)) A
```

Layer	Carrier	Semantics
MaybeT	`value: Optional[A]`	Failure propagation: BLOCKED short-circuits
WriterT	`cost: float`	Accumulated $\Delta\xi$_CP across all steps
StateT	`context: Context`	F-floor ratchet; criticality gate; step count

Monad laws in design terms:

- `return_(s) » f = f(s)` — starting fresh has no cost
- `(m » f) » g = m » (f » g)` — pipeline order is associative
- `mzero() » f = mzero()` — blocked computations stay blocked

`mplus` (<|>) — fallback: `BLOCKED <|> PASS = PASS`. Models branching design paths. The F-floor from a failed branch does NOT transfer to the fallback — the monad correctly isolates commitment state.

Key examples (ğ6 in demo):

Pipeline	Result	What it proves
`Hv1_open » meet(AtHv1_primed) » join(PsHv1)`	PASS · $\Delta\xi$=0	Cross-species conservation; Φ_c preserved through full pipeline
`strategy_A.mplus(strategy`	strategy_B	BLOCKED=PASS; F-floor isolation
`cooper_pair » tensor(TI_surface, lambda=0.3)`	PASS · Ω_Z · Φ_c	Proximity effect: topological protection and criticality propagate

---

0.10.8 ğ7 · Decompositions

Decomposition operations **invert or factor** the algebraic operations. They are the analytic tools; ğğ1–6 are the synthetic tools.

Key examples (ğ7 in demo):

cofactor — Cooper pair reverse engineering:

Operation	Category-theoretic reading	Use
`factor(s)`	Decrement morphism: step one ordinal to its sub-object	"What is the largest proper sub-synthon?"
`cofactor(C, A)`	**Inverse bifunctor**: find B s.t. tensor(A, B) ≈ C	Reverse-engineer the unknown component of a co-assembly
`kernel(s, probe)`	**Kernel of a predicate-morphism**: largest sub-s annihilated by ϕ	"What can I remove before Φ_c signal disappears?"
`principal_decomp(s)`	**Join-irreducible basis decomposition** (Birkhoff representation)	SVD analog: ordered list of primitive contributions
`project(s, dims)`	**Coordinate projection** π_S: retain S, zero out S	Subspace restriction to named primitives
`complement_rel(s, ctx, tgt)`	**Relative pseudocomplement** in a Heyting algebra	"What part of s is still needed after ctx already covers tgt?"
`primitive_peel(s, prim)`	Single-dim projection with invariant cost accounting	Remove one primitive; pay Φ_c/Ω protection costs

```
cofactor(cooper_pair, conducting_electron) -> inferred partner
    B:
  K: BOTTLENECK   -> B must be K_slow (condensate timescale,
   not Fermi velocity)
  G: CONTRIBUTOR  -> B must be G_meso (coherence length \xi ~
   100--1000 nm)
  T: PASSTHROUGH  -> B must have T_ (pairing loop)
  Phi: CONTRIBUTOR  -> B carries Phi_c (superfluid phase
   transition)
  \Omega: CONTRIBUTOR  -> B carries \Omega_Z ( winding number)
```

The cofactor reconstructs the phonon-dressed retarded partner from the observable pair. This is how Cooper pair formation is diagnosed from first principles in primitive space.

principal_decomp — SVD of GNF-2:

```
principal_decomp(GNF-2) -> 4 join-irreducible atoms:
  [1] atom[F=F_]     --- fidelity contribution    (most
   constraining)
  [2] atom[K=K_mod]   --- kinetic contribution
  [3] atom[G=G_]     --- mesoscale contribution
  [4] skeleton(GNF-2) --- categorical primitives (Phi_c,
   T_branched, ...)
```

Order = hardest to engineer first. If improving GNF-2, start at [3] (G escalation to

mesoscale is the mechanistic bottleneck for allosteric propagation).

project + complement_rel — Heyting pseudocomplement:

```
# "If the Phi/\Omega structure is already given by the
   topological material,
#  what does GNF-2 uniquely contribute toward the cooper_pair
   target?"
proj = project(cooper_pair, ["Phi", "Omega"])          # pi_{
   Phi,\Omega}(cooper_pair)
crel = complement_rel(gnf2, context=proj, target=cooper_pair)
# -> crel: K_slow + G_meso + T_branched (the part context does
    NOT cover)
```

The Heyting implication GNF-2 ⇒ cooper_pair relative to ¬context: the maximal subsynthon of GNF-2 that (a) has no overlap with the Φ/Ω projection and (b) together with it covers the target. This is **design as constructive proof**.

0.10.9 Canonical Pedagogical Example

The Webster/Tombola Hv1 trilogy (Papers 1–3) threads through every operation:

Operation	Example	Result
meet	`Hv1_open  AtHv1_primed`	Φ_c preserved; cross-species conservation
join	`Hv1_open  PsHv1`	Gymnosperm subsumed; Φ_c join-dominant
tensor	`Hv1_open` ⊗`2GBI`	T+P CONFLICT; inhibitor occludes, not merges
path	`AtHv1_silent -> AtHv1_primed`	BLOCKED; priming is a discrete T-class jump
cofactor	`cofactor(cooper_pair, electron)`	Reconstructs phonon-dressed partner
pipeline	`Hv1_open » meet » join`	Δξ=0; Φ_c throughout; monad-law compliant

The d(AtHv1_primed, PsHv1) = 0.000 result — the gymnosperm channel is algebraically identical to mechanically primed Arabidopsis — is the single number that collapses the phylogenetic argument. The RSN (Ring-Shaped Network, K_trap) is the *entire* primitive distinction between angiosperm and gymnosperm Hv channels.

---

1. **`mplus` semantics on cost**: should the fallback branch's cost be added to the primary branch's failed cost, or reset? Current design: costs accumulate (Writer is append-only). Alternative: reset on fallback (cleaner but loses failed-branch evidence).

2. **Context merging on `bind`**: `F-floor` takes max (strictest requirement wins). Is this always right for `meet` operations which lower F? Currently: `meet` does not update `context.f_floor` (it lowers F, not raises the floor). Only `join` and `HotSwap` raise the floor.
3. **`assert` timeout**: criticality probe assertions (`phi_c_score > N`) run the full Varma probe. For large catalogs this could be slow in a pipeline. Add `-timeout` flag or cache probe results per synthon.
4. **Named strategies in `.syn`**: the `strategies:` block in `.syn` allows defining reusable sub-pipelines. Should these be first-class `DesignStrategy` objects (composable with » and `<|>`) or just YAML macros (inlined at parse time)?
5. **Phase boundary semantics**: the Ward dendrogram gives an empirical boundary at d $\approx$ 9.52 for the current 8-synthon quantum catalog. Does this threshold generalise? Adding classical synthons will compress the inter-cluster gap. The open question is whether a **universal** threshold exists in the Kleisli metric, or whether boundaries are always catalog-relative. Phase 3e should test this by mixing the quantum catalog with the full classical molecular catalog and measuring whether the topological/particle split persists at the same threshold.
6. **`.syn` assert grammar extension**: predicates for Ω and quantum grammar are not yet in the dispatch table. Needed additions: `topo_index == Omega_NA`, `grammar == QUANTUM_AND`, `K_MBL`, `factor8_triggered`. These enable design programs that assert topological protection before quantum tensor operations.